Salon Shizuka — Master Engineering Reference

> Merged and reconciled from all architecture guides.

> Last updated: May 2026

> Stack: React 18 + TypeScript · Sass (7-1) · GSAP 3 · Redux Toolkit · React Hook Form · Node.js + Express · SQLite (Prisma) · LINE Messaging API · Nodemailer (Gmail OAuth2)

0. Architecture Philosophy

This project sits between a salon landing page and a transactional booking platform. The architecture should reflect that honestly — well-structured and maintainable, but not over-engineered.

Frontend pattern: Feature-Driven + Atomic UI + Service Layer.

Classic MVC is awkward in React. Instead, UI components live in Atomic Design layers (`atoms/`, `molecules/`, `organisms/`), business domains live in `features/` (each self-contained with its own components, hooks, store slice, and types), and all API calls are isolated in a `services/` layer.

Animation pattern: Centralized, not scattered.

All GSAP logic lives in `src/animations/`. Components never contain raw `gsap.to()` calls — they call named animation functions. This is the single most important structural decision for GSAP maintainability.

State pattern: Hybrid, not Redux-everywhere.

Different state has different needs. Match the tool to the job:

State Type	Tool	Reason
Booking flow (multi-step)	Redux Toolkit	Persists across steps; needs global access
Contact fields (name, email)	React Hook Form	Local form validation; no global need
Mobile menu open/close	Redux Toolkit (`uiSlice`)	Accessed from Header + overlay
Server responses / loading	Local `useState`	Simple async flag

Backend pattern: Layered Controller-Service (MVC variant).

Routes → Controllers (parse HTTP, call services) → Services (pure business logic: email, LINE, DB) → Middleware (validation, rate limiting, error handling).

1. Tech Stack Decision Matrix

Layer	Choice	Why
UI Framework	React 18 + TypeScript	Component reuse, type safety
Styling	Sass + BEM + 7-1 architecture	Scalable, no runtime cost, industry-standard
Animation	GSAP 3 + ScrollTrigger	Professional scroll animations; centralized
Global State	Redux Toolkit	Booking flow, UI state
Form State	React Hook Form + Zod resolver	Clean validation, no Redux overhead
Routing	React Router v6	Hash/anchor scrolling for SPA
Backend	Node.js + Express + TypeScript	LINE SDK support, deployment flexibility
Email	Nodemailer + Gmail OAuth2	Free, reliable — OAuth2 only (not App Password)
LINE	LINE Messaging API (push)	Owner notification; standard in Japan
Database	SQLite via Prisma ORM	Simple start; Prisma makes migration to Postgres painless
Validation	Zod (shared frontend + backend)	Single schema, no type drift
Map	Google Maps URL link	Opens native Maps app; no API key or iframe needed
Deployment	Vercel (frontend) + Railway (backend)	Free tier CI/CD; Railway avoids Render's cold starts

2. Project Structure

```
salon-shizuka/
■
■■■ apps/
■   ■■■ web/                          # React frontend (Vite + TS)
■   ■   ■■■ public/
■   ■   ■   ■■■ fonts/
■   ■   ■   ■■■ images/                # Optimized WebP assets
■   ■   ■   ■■■ favicon.ico
■   ■   ■
■   ■   ■■■ src/
■   ■       ■■■ app/                  # App-level setup
■   ■       ■   ■■■ store.ts          # RTK configureStore
■   ■       ■   ■■■ router.tsx        # React Router routes
■   ■       ■   ■■■ providers/        # <Provider>, <RouterProvider> wrappers
■   ■       ■
■   ■       ■■■ assets/
■   ■       ■   ■■■ images/
■   ■       ■   ■■■ icons/
■   ■       ■   ■■■ fonts/
■   ■       ■   ■■■ svg/
```

```
■   ■       ■
■   ■       ■■■ styles/                # 7-1 Sass architecture
■   ■       ■   ■■■ abstracts/
■   ■       ■   ■   ■■■ _variables.scss # Color, font, spacing tokens
■   ■       ■   ■   ■■■ _mixins.scss    # Breakpoints, fluid-type helpers
■   ■       ■   ■   ■■■ _functions.scss # rem(), fluid-type()
■   ■       ■   ■   ■■■ _breakpoints.scss
■   ■       ■   ■■■ base/
■   ■       ■   ■   ■■■ _reset.scss
■   ■       ■   ■   ■■■ _typography.scss
■   ■       ■   ■   ■■■ _globals.scss
■   ■       ■   ■   ■■■ _animations.scss # Keyframe definitions only
■   ■       ■   ■■■ layout/
■   ■       ■   ■   ■■■ _header.scss
■   ■       ■   ■   ■■■ _footer.scss
■   ■       ■   ■   ■■■ _grid.scss
■   ■       ■   ■■■ components/
■   ■       ■   ■   ■■■ _button.scss
■   ■       ■   ■   ■■■ _card.scss
■   ■       ■   ■   ■■■ _modal.scss
■   ■       ■   ■   ■■■ _calendar.scss
■   ■       ■   ■■■ pages/
■   ■       ■   ■   ■■■ _home.scss
■   ■       ■   ■   ■■■ _reservation.scss
■   ■       ■   ■■■ main.scss          # Imports everything
■   ■       ■
■   ■       ■■■ components/            # Atomic Design — pure, reusable UI
■   ■       ■   ■■■ atoms/
■   ■       ■   ■   ■■■ Button/
■   ■       ■   ■   ■   ■■■ Button.tsx
■   ■       ■   ■   ■   ■■■ Button.module.scss
■   ■       ■   ■   ■■■ Input/
■   ■       ■   ■   ■■■ Label/
■   ■       ■   ■   ■■■ Icon/
■   ■       ■   ■   ■■■ Logo/
■   ■       ■   ■   ■■■ Tag/
■   ■       ■   ■■■ molecules/
■   ■       ■   ■   ■■■ FormField/      # Label + Input + error message
■   ■       ■   ■   ■■■ NavLink/
■   ■       ■   ■   ■■■ PriceRow/
■   ■       ■   ■   ■■■ TimeSlot/
■   ■       ■   ■   ■■■ ServiceCard/
■   ■       ■   ■   ■■■ MenuCard/
■   ■       ■   ■   ■■■ ReviewCard/
■   ■       ■   ■■■ organisms/
■   ■       ■   ■   ■■■ Header/
■   ■       ■   ■   ■   ■■■ Header.tsx
■   ■       ■   ■   ■   ■■■ Header.module.scss
■   ■       ■   ■   ■   ■■■ MobileMenu/ # Full-screen overlay
■   ■       ■   ■   ■■■ Hero/
■   ■       ■   ■   ■■■ ConceptSection/
■   ■       ■   ■   ■■■ GallerySlider/
■   ■       ■   ■   ■■■ OwnerSection/
■   ■       ■   ■   ■■■ StyleGuideTeaser/
■   ■       ■   ■   ■■■ MenuSection/
■   ■       ■   ■   ■■■ VoiceSection/
■   ■       ■   ■   ■■■ NewsletterBanner/
■   ■       ■   ■   ■■■ ReservationForm/
■   ■       ■   ■   ■■■ MapHours/
■   ■       ■   ■   ■■■ Footer/
■   ■       ■   ■■■ templates/
■   ■       ■       ■■■ MainLayout/     # Assembles all sections
■   ■       ■
■   ■       ■■■ features/              # Business domains — self-contained modules
■   ■       ■   ■■■ booking/
■   ■       ■   ■   ■■■ components/     # Calendar, TimeSlotPicker, ServiceGrid
■   ■       ■   ■   ■■■ hooks/          # useBookingForm.ts
■   ■       ■   ■   ■■■ store/
■   ■       ■   ■   ■   ■■■ bookingSlice.ts
■   ■       ■   ■   ■   ■■■ bookingSelectors.ts
```

```
│   │       │   │   ├── types/
│   │       │   │   │   └── booking.types.ts
│   │       │   │   └── utils/
│   │       │   │       └── formatBookingSummary.ts
│   │       │   ├── newsletter/
│   │       │   │   ├── components/
│   │       │   │   ├── hooks/
│   │       │   │   ├── store/
│   │       │   │   │   └── newsletterSlice.ts
│   │       │   ├── menu/
│   │       │   ├── gallery/
│   │       │
│   │       ├── animations/            # ALL GSAP logic lives here — never in components
│   │       │   ├── gsap/
│   │       │   │   ├── hero.ts         # heroReveal(), heroTextSplit()
│   │       │   │   ├── fade.ts         # fadeInUp(), fadeInStagger()
│   │       │   │   ├── scrollReveal.ts # sectionLabelReveal(), imageReveal()
│   │       │   │   ├── gallery.ts      # gallerySlide(), imageScale()
│   │       │   │   ├── mobileMenu.ts   # menuOverlayIn(), menuOverlayOut()
│   │       │   ├── hooks/
│   │       │   │   ├── useScrollReveal.ts
│   │       │   │   ├── useHeroAnimation.ts
│   │       │
│   │       ├── services/              # API call layer (Axios)
│   │       │   ├── api.ts             # Axios instance with base URL + interceptors
│   │       │   ├── bookingService.ts
│   │       │   ├── newsletterService.ts
│   │       │
│   │       ├── hooks/                 # Shared custom hooks
│   │       │   ├── useBreakpoint.ts
│   │       │   ├── useOpenNow.ts       # Computes open/closed from hours table
│   │       │
│   │       ├── utils/
│   │       │   ├── formatDate.ts
│   │       │   ├── formatPrice.ts      # ¥ formatting
│   │       │   ├── constants.ts        # Service list, time slots, prices
│   │       │
│   │       ├── types/
│   │       │   ├── service.types.ts
│   │       │
│   │       ├── App.tsx
│   │       ├── main.tsx
│   │
│   ├── api/                           # Express backend (Node.js + TS)
│   │   ├── src/
│   │   │   ├── config/
│   │   │   │   ├── env.ts             # Typed environment variable loader
│   │   │   │   ├── line.ts            # LINE client config
│   │   │   ├── controllers/
│   │   │   │   ├── booking.controller.ts
│   │   │   │   ├── newsletter.controller.ts
│   │   │   ├── services/
│   │   │   │   ├── email.service.ts    # Nodemailer Gmail OAuth2
│   │   │   │   ├── line.service.ts     # LINE push message
│   │   │   │   ├── booking.service.ts  # DB write + notification orchestration
│   │   │   │   ├── logger.service.ts   # Structured request + error logging
│   │   │   ├── routes/
│   │   │   │   ├── booking.routes.ts
│   │   │   │   ├── newsletter.routes.ts
│   │   │   ├── middleware/
│   │   │   │   ├── validateBooking.ts  # Zod schema guard
│   │   │   │   ├── rateLimiter.ts      # express-rate-limit
│   │   │   │   ├── errorHandler.ts     # Global error middleware
│   │   │   │   ├── cors.ts
│   │   │   ├── db/
│   │   │   │   ├── client.ts           # Prisma client singleton
│   │   │   │   ├── schema.prisma       # Booking + Newsletter models
│   │   │   ├── types/
│   │   │   │   ├── booking.types.ts
│   │   │   ├── app.ts
```

```
■
■■■ shared/                              # Shared between frontend and backend
■   ■■■ schemas/
■   ■   ■■■ bookingSchema.ts             # Zod — single source of truth
■   ■   ■■■ newsletterSchema.ts
■   ■■■ types/
■   ■   ■■■ booking.types.ts             # Inferred from Zod schemas
■   ■   ■■■ service.types.ts
■   ■■■ constants/
■       ■■■ services.ts                  # Service list with prices
■       ■■■ timeSlots.ts
■
■■■ docs/
■   ■■■ architecture/
■   ■   ■■■ overview.md
■   ■   ■■■ state-strategy.md
■   ■   ■■■ animation-guidelines.md
■   ■■■ ui-ux/
■   ■   ■■■ design-tokens.md
■   ■   ■■■ responsive-strategy.md
■   ■   ■■■ component-map.md
■   ■■■ backend/
■   ■   ■■■ api-reference.md
■   ■   ■■■ notification-flow.md
■   ■   ■■■ database-schema.md
■   ■■■ deployment/
■       ■■■ checklist.md
■
■■■ .github/
■   ■■■ workflows/
■       ■■■ ci.yml                       # Type check + lint on push
■
■■■ .env.example
■■■ .env.local                           # Local overrides (gitignored)
■■■ .env.development
■■■ .env.production
■■■ .gitignore
■■■ package.json                         # Root workspace scripts
■■■ README.md
```

3. Design Tokens (`_variables.scss`)

```scss
// ■■■ Colors ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
$color-bg:          #FAF5EF;   // Warm cream
$color-text:        #3D3733;   // Dark warm brown
$color-text-muted:  #7A6E66;
$color-accent:      #C9A96E;   // Gold
$color-accent-dark: #A8834A;
$color-white:       #FFFFFF;
$color-border:      #E8DDD2;

// ■■■ Typography ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
$font-serif:    'Cormorant Garamond', 'Noto Serif JP', serif;
$font-sans:     'Jost', 'Noto Sans JP', sans-serif;
$font-logo:     'Great Vibes', cursive;

// ■■■ Spacing scale ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
$space-xs:   0.5rem;
$space-sm:   1rem;
$space-md:   2rem;
$space-lg:   4rem;
$space-xl:   8rem;

// ■■■ Breakpoints ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
$bp-mobile:  375px;
```

```
$bp-tablet:  768px;
$bp-laptop:  1024px;
$bp-desktop: 1280px;

// ■■■ Z-index ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
$z-base:     1;
$z-overlay:  100;
$z-nav:      200;
$z-modal:    300;
```

4. State Management

Redux Toolkit Slices

Only two slices belong in Redux: booking flow state and global UI state (mobile menu). Everything else stays local.

```
// features/booking/store/bookingSlice.ts
import { createSlice, PayloadAction } from '@reduxjs/toolkit';

interface ServiceItem {
  id: string;
  name: string;
  price: number;
}

interface BookingState {
  selectedDate: string | null;
  selectedTime: string | null;
  selectedServices: ServiceItem[];
  selectedAddons: ServiceItem[];
  // Contact fields are NOT stored here — React Hook Form owns them
}

const initialState: BookingState = {
  selectedDate: null,
  selectedTime: null,
  selectedServices: [],
  selectedAddons: [],
};

export const bookingSlice = createSlice({
  name: 'booking',
  initialState,
  reducers: {
    setDate: (state, action: PayloadAction<string>) => {
      state.selectedDate = action.payload;
    },
    setTime: (state, action: PayloadAction<string>) => {
      state.selectedTime = action.payload;
    },
    toggleService: (state, action: PayloadAction<ServiceItem>) => {
      const exists = state.selectedServices.find(s => s.id === action.payload.id);
      if (exists) {
        state.selectedServices = state.selectedServices.filter(s => s.id !== action.payload.id);
      } else {
        state.selectedServices.push(action.payload);
      }
    },
    toggleAddon: (state, action: PayloadAction<ServiceItem>) => {
      const exists = state.selectedAddons.find(a => a.id === action.payload.id);
      if (exists) {
        state.selectedAddons = state.selectedAddons.filter(a => a.id !== action.payload.id);
```

```
      } else {
        state.selectedAddons.push(action.payload);
      }
    },
    resetBooking: () => initialState,
  },
});

// features/booking/store/bookingSelectors.ts
import { createSelector } from '@reduxjs/toolkit';
import type { RootState } from '../../../app/store';

const selectBooking = (state: RootState) => state.booking;

export const selectTotal = createSelector(
  [selectBooking],
  (booking) =>
    [...booking.selectedServices, ...booking.selectedAddons]
      .reduce((sum, item) => sum + item.price, 0)
);

// app/store/slices/uiSlice.ts
import { createSlice } from '@reduxjs/toolkit';

const uiSlice = createSlice({
  name: 'ui',
  initialState: { mobileMenuOpen: false },
  reducers: {
    openMobileMenu: (state) => { state.mobileMenuOpen = true; },
    closeMobileMenu: (state) => { state.mobileMenuOpen = false; },
    toggleMobileMenu: (state) => { state.mobileMenuOpen = !state.mobileMenuOpen; },
  },
});
```

React Hook Form — Contact Step

The final name/email step uses React Hook Form with a Zod resolver — cleaner validation, no Redux overhead.

```
// features/booking/hooks/useBookingForm.ts
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { contactSchema } from '../../../../shared/schemas/bookingSchema';
import type { z } from 'zod';

type ContactFields = z.infer<typeof contactSchema>;

export const useBookingForm = () => {
  return useForm<ContactFields>({
    resolver: zodResolver(contactSchema),
    defaultValues: { name: '', email: '' },
  });
};
```

5. Shared Schemas (`shared/schemas/`)

The `shared/` package is the single source of truth for Zod schemas and TypeScript types. Both the frontend and backend import from here — no duplication, no type drift.

```
// shared/schemas/bookingSchema.ts
import { z } from 'zod';
// Used by React Hook Form on the frontend
```

```
export const contactSchema = z.object({
  name:  z.string().min(1, '[redacted]').max(100),
  email: z.string().email('[redacted]'),
});

// Full booking schema used by Express + Zod middleware
export const bookingSchema = z.object({
  name:     z.string().min(1).max(100),
  email:    z.string().email(),
  date:     z.string().regex(/^\d{4}-\d{2}-\d{2}$/, 'Invalid date format'),
  time:     z.string().regex(/^\d{2}:\d{2}$/, 'Invalid time format'),
  services: z.array(z.object({
    id:    z.string(),
    name:  z.string(),
    price: z.number().min(0),
  })).min(1, '[redacted]'),
  addons: z.array(z.object({
    id:    z.string(),
    name:  z.string(),
    price: z.number().min(0),
  })),
  // total is NOT accepted from client — always recalculated server-side
});

export type BookingPayload = z.infer<typeof bookingSchema>;

// shared/schemas/newsletterSchema.ts
import { z } from 'zod';

export const newsletterSchema = z.object({
  email:  z.string().email(),
  source: z.string().optional().default('website'),
});

export type NewsletterPayload = z.infer<typeof newsletterSchema>;
```

6. Animation System

All GSAP logic is isolated in `src/animations/`. Components receive animation functions via hooks — they never call GSAP directly.

```
// animations/gsap/fade.ts
import { gsap } from 'gsap';
import { ScrollTrigger } from 'gsap/ScrollTrigger';

gsap.registerPlugin(ScrollTrigger);

export const fadeInUp = (el: HTMLElement, delay = 0) => {
  return gsap.fromTo(el,
    { opacity: 0, y: 40 },
    {
      opacity: 1, y: 0,
      duration: 0.8,
      delay,
      ease: 'power2.out',
      scrollTrigger: { trigger: el, start: 'top 85%' },
    }
  );
};

export const fadeInStagger = (els: HTMLElement[], stagger = 0.08) => {
  return gsap.fromTo(els,
    { opacity: 0, y: 30 },
    {
```

```
      opacity: 1, y: 0,
      duration: 0.7,
      ease: 'power2.out',
      stagger,
      scrollTrigger: { trigger: els[0], start: 'top 85%' },
    }
  );
};

// animations/gsap/mobileMenu.ts
import { gsap } from 'gsap';

export const menuOverlayIn = (overlay: HTMLElement, navItems: HTMLElement[]) => {
  const tl = gsap.timeline();
  tl.fromTo(overlay,
    { opacity: 0, y: -20 },
    { opacity: 1, y: 0, duration: 0.4, ease: 'power2.out' }
  );
  tl.fromTo(navItems,
    { opacity: 0, y: 16 },
    { opacity: 1, y: 0, duration: 0.3, stagger: 0.06, ease: 'power2.out' },
    '-=0.2'
  );
  return tl;
};

export const menuOverlayOut = (overlay: HTMLElement) => {
  return gsap.to(overlay, { opacity: 0, y: -20, duration: 0.3, ease: 'power2.in' });
};

// animations/gsap/hero.ts
import { gsap } from 'gsap';
import { SplitText } from 'gsap/SplitText'; // GSAP Club or use manual split

export const heroReveal = (headline: HTMLElement, subtext: HTMLElement) => {
  const tl = gsap.timeline();
  const split = new SplitText(headline, { type: 'chars' });

  tl.fromTo(split.chars,
    { opacity: 0, y: 20 },
    { opacity: 1, y: 0, duration: 0.6, stagger: 0.03, ease: 'power3.out' }
  );
  tl.fromTo(subtext,
    { opacity: 0, y: 16 },
    { opacity: 1, y: 0, duration: 0.6, ease: 'power2.out' },
    '-=0.3'
  );
  return tl;
};

// animations/hooks/useScrollReveal.ts
import { useEffect, useRef } from 'react';
import { fadeInUp } from '../gsap/fade';
import { ScrollTrigger } from 'gsap/ScrollTrigger';

export const useScrollReveal = <T extends HTMLElement>() => {
  const ref = useRef<T>(null);

  useEffect(() => {
    const el = ref.current;
    if (!el) return;
    const anim = fadeInUp(el);
    return () => {
      anim.scrollTrigger?.kill();
      ScrollTrigger.getAll().forEach(t => t.kill());
    };
  }, []);

  return ref;
```

```
};
```

GSAP Animation Plan

Element	Function	Trigger
Hero headline	`heroReveal()` — char stagger	Page load
Hero subtext	`heroReveal()` — fade up	Page load, after headline
Section labels ("CONCEPT", "MENU"…)	`sectionLabelReveal()` — letter-spacing expand	ScrollTrigger 80%
Section headings (Japanese)	`fadeInUp()`	ScrollTrigger 85%
Gallery images	`imageScale()` — scale 0.95→1 + fade, stagger	ScrollTrigger
Menu rows	`fadeInStagger()` — slide from left	ScrollTrigger
Review card	`fadeInUp()` + slight scale	ScrollTrigger
Mobile menu overlay	`menuOverlayIn()` / `menuOverlayOut()`	Button click
Reservation success	`fadeInUp()` on confirmation block	Form submit

7. Backend — Express API

Layered Architecture

```
Request → Route → Middleware (validate, rate-limit) → Controller → Service → Response
                                                                        ↓
                                                                DB · Email · LINE
```

Routes

```
POST /api/booking       → create reservation + notify
POST /api/newsletter    → register email subscription
```

Database Schema (Prisma)

```
// apps/api/src/db/schema.prisma
datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model Booking {
  id        String   @id @default(cuid())
```

```
  name      String
  email     String
  date      String
  time      String
  services  Json     // Array of { id, name, price }
  addons    Json
  total     Int      // Recalculated server-side, stored in yen
  createdAt DateTime @default(now())
}

model Newsletter {
  id        String   @id @default(cuid())
  email     String   @unique
  source    String   @default("website")
  createdAt DateTime @default(now())
}
```

> Start with SQLite — zero config, no server. When reservation volume grows, change `provider = "postgresql"` in one line. Prisma handles the rest.

Booking Controller

```
// controllers/booking.controller.ts
import type { Request, Response, NextFunction } from 'express';
import { bookingService } from '../services/booking.service';
import { logger } from '../services/logger.service';

export const createBooking = async (req: Request, res: Response, next: NextFunction) => {
  try {
    await bookingService.create(req.body);
    res.status(200).json({ success: true });
  } catch (err) {
    logger.error('Booking creation failed', { error: err, body: req.body });
    next(err);
  }
};
```

Booking Service — Orchestration

```
// services/booking.service.ts
import { prisma } from '../db/client';
import { sendOwnerNotification, sendGuestConfirmation } from './email.service';
import { pushLineMessage } from './line.service';
import type { BookingPayload } from '../../../shared/schemas/bookingSchema';

export const bookingService = {
  async create(data: BookingPayload) {
    // 1. Recalculate total server-side — never trust client value
    const total = [...data.services, ...data.addons]
      .reduce((sum, item) => sum + item.price, 0);

    // 2. Persist to database
    const booking = await prisma.booking.create({
      data: { ...data, total },
    });

    // 3. Fire all notifications in parallel — don't block on each other
    await Promise.allSettled([
      sendOwnerNotification(booking),
      sendGuestConfirmation(booking),
      pushLineMessage(booking),
    ]);
    // allSettled: if LINE fails, email still goes out. Log failures separately.

    return booking;
  },
```

```
};
```

> Note: `Promise.allSettled` instead of `Promise.all` — if LINE is temporarily down, the email confirmation still reaches the guest. Failures are logged, not thrown.

Email Service (Nodemailer + Gmail OAuth2)

```
// services/email.service.ts
import nodemailer from 'nodemailer';
import { env } from '../config/env';
import type { Booking } from '@prisma/client';

const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    type: 'OAuth2',
    user: env.GMAIL_USER,
    clientId: env.GMAIL_CLIENT_ID,
    clientSecret: env.GMAIL_CLIENT_SECRET,
    refreshToken: env.GMAIL_REFRESH_TOKEN,
  },
});

export async function sendOwnerNotification(booking: Booking) {
  await transporter.sendMail({
    from: env.GMAIL_USER,
    to: env.OWNER_EMAIL,
    subject: `■■■■■■■${booking.name} ■ — ${booking.date} ${booking.time}`,
    html: `
      <h3>■■■■■■■■■■■</h3>
      <p><b>■■■:</b> ${booking.name} ■</p>
      <p><b>■■:</b> ${booking.date} ${booking.time}</p>
      <p><b>■■:</b> ¥${booking.total.toLocaleString()}</p>
      <p><b>Email:</b> ${booking.email}</p>
    `,
  });
}

export async function sendGuestConfirmation(booking: Booking) {
  await transporter.sendMail({
    from: `"SALON SHIZUKA" <${env.GMAIL_USER}>`,
    to: booking.email,
    subject: `■■■■■■■■SALON SHIZUKA — ${booking.date}`,
    html: `
      <p>${booking.name} ■</p>
      <p>■■■■■■■■■■■■■■■</p>
      <p><b>■■:</b> ${booking.date} ${booking.time}</p>
      <p>24■■■■■■■■■■■■■■■</p>
      <p>— SALON SHIZUKA</p>
    `,
  });
}
```

> Gmail OAuth2 setup: Google Cloud Console → Create OAuth2 credentials → obtain refresh token via [OAuth Playground](https://developers.google.com/oauthplayground). Scope: `https://mail.google.com/`. Do NOT use App Passwords — Google is deprecating them and they are weaker.

LINE Service

```
// services/line.service.ts
import axios from 'axios';
import { env } from '../config/env';
import type { Booking } from '@prisma/client';
export async function pushLineMessage(booking: Booking) {
```

```
  const text = [
    '■ ■■■■■■■■■■',
    `■■■: ${booking.name} ■`,
    `■■: ${booking.date} ${booking.time}`,
    `■■: ¥${booking.total.toLocaleString()}`,
    `Email: ${booking.email}`,
  ].join('\n');

  await axios.post(
    'https://api.line.me/v2/bot/message/push',
    {
      to: env.LINE_OWNER_USER_ID,
      messages: [{ type: 'text', text }],
    },
    {
      headers: { Authorization: `Bearer ${env.LINE_CHANNEL_ACCESS_TOKEN}` },
    }
  );
}
```

> LINE setup: LINE Developers Console → Messaging API channel → Channel Access Token → owner adds bot as friend → get owner's LINE User ID (starts with `U`) via the webhook or `/v2/profile` API.

Typed Environment Config

```
// config/env.ts
import { z } from 'zod';
import dotenv from 'dotenv';
dotenv.config();

const envSchema = z.object({
  PORT:                     z.string().default('4000'),
  CORS_ORIGIN:              z.string(),
  DATABASE_URL:             z.string(),
  GMAIL_USER:               z.string(),
  GMAIL_CLIENT_ID:          z.string(),
  GMAIL_CLIENT_SECRET:      z.string(),
  GMAIL_REFRESH_TOKEN:      z.string(),
  OWNER_EMAIL:              z.string().email(),
  LINE_CHANNEL_ACCESS_TOKEN: z.string(),
  LINE_OWNER_USER_ID:       z.string(),
  NEWSLETTER_NOTIFY_EMAIL:  z.string().email(),
});

export const env = envSchema.parse(process.env);
// Throws at startup if any required variable is missing — fail fast, not at runtime.
```

Logger Service

```
// services/logger.service.ts
// Simple structured logger — swap for winston/pino later if needed
export const logger = {
  info: (msg: string, meta?: object) =>
    console.log(JSON.stringify({ level: 'info', msg, ...meta, ts: new Date().toISOString() })),
  error: (msg: string, meta?: object) =>
    console.error(JSON.stringify({ level: 'error', msg, ...meta, ts: new Date().toISOString() })),
};
```

Zod Validation Middleware

```
// middleware/validateBooking.ts
import type { Request, Response, NextFunction } from 'express';
import { bookingSchema } from '../../../shared/schemas/bookingSchema';

export const validateBooking = (req: Request, res: Response, next: NextFunction) => {
```

```
  const result = bookingSchema.safeParse(req.body);
  if (!result.success) {
    return res.status(400).json({
      error: 'Validation failed',
      issues: result.error.flatten(),
    });
  }
  req.body = result.data; // Replace with parsed, typed data
  next();
};
```

Global Error Handler

```
// middleware/errorHandler.ts
import type { Request, Response, NextFunction } from 'express';
import { logger } from '../services/logger.service';

export const errorHandler = (err: Error, _req: Request, res: Response, _next: NextFunction) => {
  logger.error('Unhandled error', { message: err.message, stack: err.stack });
  res.status(500).json({ error: 'Internal server error' });
};
```

Rate Limiting

```
// middleware/rateLimiter.ts
import rateLimit from 'express-rate-limit';

export const bookingLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5,
  message: { error: 'Too many booking attempts. Please try again later.' },
});

export const newsletterLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1 hour
  max: 3,
  message: { error: 'Too many signup attempts.' },
});
```

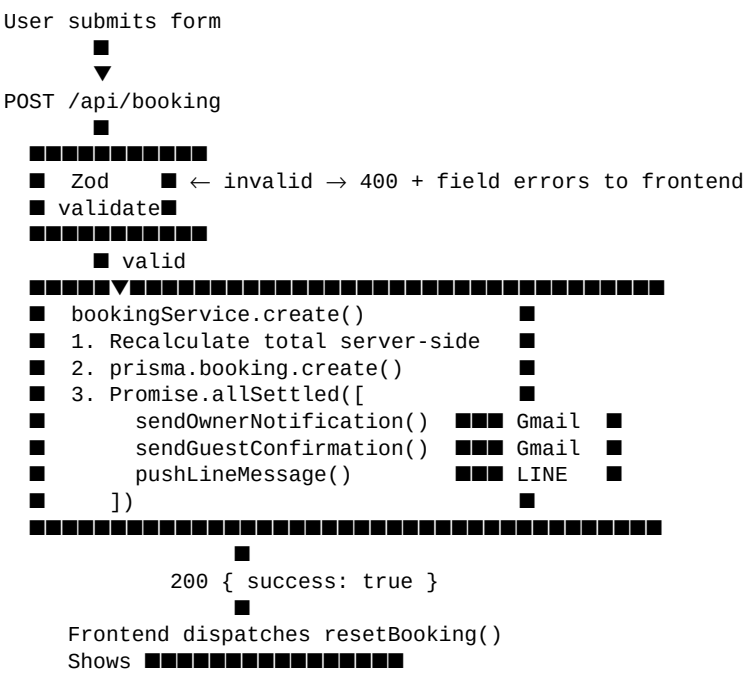
8. Map Implementation

```
// Simple, reliable — opens native Maps app on iOS/Android automatically
const MAP_URL = 'https://www.google.com/maps/search/?api=1&query=[redacted]1-1-1';

<Button
  as="a"
  href={MAP_URL}
  target="_blank"
  rel="noopener noreferrer"
  icon={<MapPinIcon />}
>
  OPEN MAP
</Button>
```

No user-agent sniffing needed. Google Maps URLs open the native app on mobile automatically.

9. Notification Flow



10. Environment Variables

```
# .env.example — copy to .env.local, never commit secrets

# Frontend (Vite — must be prefixed VITE_)
VITE_API_BASE_URL=http://localhost:4000

# Backend
PORT=4000
CORS_ORIGIN=http://localhost:5173
DATABASE_URL=file:./dev.db

# Gmail OAuth2
GMAIL_USER=your@gmail.com
GMAIL_CLIENT_ID=
GMAIL_CLIENT_SECRET=
GMAIL_REFRESH_TOKEN=
OWNER_EMAIL=owner@example.com

# LINE Messaging API
LINE_CHANNEL_ACCESS_TOKEN=
LINE_OWNER_USER_ID=          # Starts with U (e.g. U1234abc...)

# Newsletter
NEWSLETTER_NOTIFY_EMAIL=owner@example.com
```

Maintain three environment files: `.env.local` for dev secrets, `.env.development` for non-secret dev config, `.env.production` for production values set in your deployment dashboard (Railway/Vercel). Only `.env.example` is committed to git.

11. Package Dependencies

Frontend (`apps/web/package.json`)

```
{
  "dependencies": {
    "react": "^18.3.0",
    "react-dom": "^18.3.0",
    "react-router-dom": "^6.24.0",
    "@reduxjs/toolkit": "^2.3.0",
    "react-redux": "^9.1.0",
    "react-hook-form": "^7.52.0",
    "@hookform/resolvers": "^3.6.0",
    "gsap": "^3.12.5",
    "axios": "^1.7.0",
    "sass": "^1.77.0",
    "zod": "^3.23.0"
  },
  "devDependencies": {
    "typescript": "^5.5.0",
    "vite": "^5.3.0",
    "@vitejs/plugin-react": "^4.3.0",
    "@types/react": "^18.3.0",
    "eslint": "^9.0.0"
  }
}
```

Backend (`apps/api/package.json`)

```
{
  "dependencies": {
    "express": "^4.19.0",
    "nodemailer": "^6.9.0",
    "axios": "^1.7.0",
    "zod": "^3.23.0",
    "cors": "^2.8.5",
    "express-rate-limit": "^7.3.0",
    "dotenv": "^16.4.0",
    "@prisma/client": "^5.16.0"
  },
  "devDependencies": {
    "typescript": "^5.5.0",
    "ts-node-dev": "^2.0.0",
    "prisma": "^5.16.0",
    "@types/express": "^4.17.0",
    "@types/nodemailer": "^6.4.0",
    "@types/cors": "^2.8.0"
  }
}
```

12. Development Setup

Prerequisites

- Node.js 20+

- npm 10+ or pnpm

Bootstrap

```
git clone https://github.com/yourname/salon-shizuka.git
cd salon-shizuka
# Install root workspace dependencies
```

```
npm install

# Frontend
cd apps/web
npm install
npm run dev         # → http://localhost:5173

# Backend (new terminal)
cd apps/api
npm install
cp ../../.env.example .env.local    # Fill in credentials
npx prisma generate
npx prisma db push                  # Creates dev.db
npm run dev                         # → http://localhost:4000
```

13. Build Order (Section by Section)

Build in this order — each section is independently testable before the next begins. Add GSAP animations only after the layout is stable (Phase 3).

| Phase | # | Section | Component | Key complexity |

|---|---|---|---|---|

| 1 — Foundation | — | Sass tokens, store, router | — | Design tokens, RTK setup, RHF install |

| 2 — Static UI | 1 | Header + Nav | `Header`, `MobileMenu` | Redux `uiSlice`, hamburger |

| | 2 | Hero | `Hero` | CTA button, background image |

| | 3 | Concept | `ConceptSection` | Image + text layout |

| | 4 | Gallery | `GallerySlider` | Image grid placeholder |

| | 5 | Owner bio | `OwnerSection` | Simple layout |

| | 6 | Style Guide | `StyleGuideTeaser` | Image grid |

| | 7 | Menu / Pricing | `MenuSection` | Accordion, price formatting |

| | 8 | Voice / Reviews | `VoiceSection` | Star atoms, review card |

| | 9 | Newsletter | `NewsletterBanner` | RHF email field |

| | 10 | Map & Hours | `MapHours` | Hours table, open-now badge |

| | 11 | Footer | `Footer` | Social + nav links |

| 3 — GSAP | — | All sections | `animations/` | Hero reveal, scroll reveals, menu overlay |

| 4 — Reservation | 12 | Reservation | `ReservationForm` | Calendar, time slots, Redux, RHF contact step |

| 5 — Backend | — | API + DB | `apps/api` | Prisma, email, LINE |

| 6 — Production | — | Optimization | — | a11y, SEO, image opt, security |

14. Responsive Strategy

Mobile is NOT a compressed desktop. Each breakpoint has a distinct UX goal:

| Device | UX Goal | Key differences |

|---|---|---|

| Mobile (375px) | Fast booking funnel | Single column, large tap targets, sticky CTA |

| Tablet (768px) | Comfortable reading | Two-column layouts, gallery grid |

| Laptop (1024px) | Editorial experience | Full horizontal nav, side-by-side reservation layout |

| Desktop (1280px) | Maximum whitespace | Generous `$space-xl` padding, max-width container |

15. Deployment

Frontend → Vercel

```
# Vercel dashboard settings:
# Root directory:   apps/web
# Build command:    npm run build
# Output dir:       dist
# Env var:          VITE_API_BASE_URL=https://your-api.railway.app
```

Backend → Railway

```
# Railway dashboard settings:
# Root:             apps/api
# Start command:    npm run start
# Add all .env vars in the Railway Variables panel
# Railway provides HTTPS URL → set as CORS_ORIGIN in frontend env
```

> Use Railway over Render for the backend. Render's free tier cold-starts after 15 minutes of inactivity — unacceptable for a booking form where the owner expects instant LINE and email notifications.

16. Security Checklist

- [ ] No secrets in frontend code — all tokens backend-only

- [ ] Rate-limit `/api/booking` — 5 requests / 15 min / IP

- [ ] Rate-limit `/api/newsletter` — 3 requests / hour / IP

- [ ] CORS locked to your frontend domain in production

- [ ] `bookingSchema` validates all input server-side

- [ ] `total` always recalculated server-side from `services` array — never trust client value

- [ ] Honeypot field in reservation form (hidden input, reject if filled)

- [ ] `env.ts` throws at startup if any required variable is missing

- [ ] `.env*` files in `.gitignore` (except `.env.example`)

- [ ] Gmail uses OAuth2, not App Password

17. Content Assets Checklist

Prepare these before coding the sections that need them:

Asset	Format	Notes
Hero background	WebP, 1440w / 768w / 375w	Use `<picture>` srcset
Salon gallery photos	WebP, max 800px wide	4 – 6 images
Owner photo (shizuka)	WebP, square crop	Concept + owner sections
Style guide hair photos	WebP	4 images for grid
Review customer avatar	WebP, circle crop	M.T.
Payment method icons	SVG	Cash, card, QR
Social icons	SVG inline	Facebook, Instagram, X, LinkedIn, YouTube

18. Recommended Google Fonts

```
<!-- index.html -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?
  family=Cormorant+Garamond:ital,wght@0,400;0,500;0,600;1,400&
  family=Great+Vibes&
  family=Jost:wght@300;400;500&
  family=Noto+Serif+JP:wght@400;500&
  family=Noto+Sans+JP:wght@300;400;500&
  display=swap" rel="stylesheet">
```

This document is the single authoritative engineering reference for Salon Shizuka. Do not maintain parallel guides — update this one.